

hkw2/sh2_packages.vhd

```

1  -----
2  --
3  --  SH-2 Control Unit
4  --
5  --  This is a package with all the instruction constants for the SH-2. These are
6  --  used in the control unit to generate the appropriate control signals for each instruction.
7  --  They are defined as constants with the appropriate bits set and don't-cares
8  --  where the instruction fields vary. MAC, MUL, DIV, and other instructions that
9  --  are not implementing are not included.
10 --
11 --  Packages included are:
12 --      InstructionConstants - the instruction constants
13 --
14 --  Revision History:
15 --      26 Apr 26  Simone Shevchuk      Initial revision.
16 --      28 Apr 26  Simone Shevchuk      Add constants for common load values
17 --      30 Apr 26  Simone Shevchuk      Add type records to unpack in CPU
18 -----
19
20 library ieee;
21 use ieee.std_logic_1164.all;
22
23 package InstructionConstants is
24
25     -- =====
26     -- Opcode pattern constants (don't-cares where reg or imm)
27     -- =====
28
29     constant RESET_VECTOR_AT_0          : std_logic_vector(15 downto 0) := "0000000000000000";
30     constant MOV_IMM_IMM_RN            : std_logic_vector(15 downto 0) := "1110-----";
31     constant MOV_RM_RN                 : std_logic_vector(15 downto 0) := "0110-----0011";
32     constant MOV_W_AT_DISP_PC_RN       : std_logic_vector(15 downto 0) := "1001-----";
33     constant MOV_L_AT_DISP_PC_RN       : std_logic_vector(15 downto 0) := "1101-----";
34     constant MOV_B_RM_AT_RN            : std_logic_vector(15 downto 0) := "0010-----0000";
35     constant MOV_W_RM_AT_RN            : std_logic_vector(15 downto 0) := "0010-----0001";
36     constant MOV_L_RM_AT_RN            : std_logic_vector(15 downto 0) := "0010-----0010";
37     constant MOV_B_AT_RM_RN            : std_logic_vector(15 downto 0) := "0110-----0000";
38     constant MOV_W_AT_RM_RN            : std_logic_vector(15 downto 0) := "0110-----0001";
39     constant MOV_L_AT_RM_RN            : std_logic_vector(15 downto 0) := "0110-----0010";
40     constant MOV_B_AT_RM_PLUS_RN        : std_logic_vector(15 downto 0) := "0110-----0100";
41     constant MOV_W_AT_RM_PLUS_RN        : std_logic_vector(15 downto 0) := "0110-----0101";
42     constant MOV_L_AT_RM_PLUS_RN        : std_logic_vector(15 downto 0) := "0110-----0110";
43     constant MOV_B_RM_AT_MINUS_RN       : std_logic_vector(15 downto 0) := "0010-----0100";
44     constant MOV_W_RM_AT_MINUS_RN       : std_logic_vector(15 downto 0) := "0010-----0101";
45     constant MOV_L_RM_AT_MINUS_RN       : std_logic_vector(15 downto 0) := "0010-----0110";
46     constant MOV_B_R0_AT_DISP_RN        : std_logic_vector(15 downto 0) := "10000000-----";
47     constant MOV_W_R0_AT_DISP_RN        : std_logic_vector(15 downto 0) := "10000001-----";
48     constant MOV_L_R0_AT_DISP_RN        : std_logic_vector(15 downto 0) := "0001-----";
49     constant MOV_B_AT_DISP_RM_R0        : std_logic_vector(15 downto 0) := "10000100-----";
50     constant MOV_W_AT_DISP_RM_R0        : std_logic_vector(15 downto 0) := "10000101-----";
51     constant MOV_L_AT_DISP_RM_RN        : std_logic_vector(15 downto 0) := "0101-----";
52     constant MOV_B_RM_AT_R0_RN          : std_logic_vector(15 downto 0) := "0000-----0100";
53     constant MOV_W_RM_AT_R0_RN          : std_logic_vector(15 downto 0) := "0000-----0101";
54     constant MOV_L_RM_AT_R0_RN          : std_logic_vector(15 downto 0) := "0000-----0110";
55     constant MOV_B_AT_R0_RM_RN          : std_logic_vector(15 downto 0) := "0000-----1100";
56     constant MOV_W_AT_R0_RM_RN          : std_logic_vector(15 downto 0) := "0000-----1101";
57     constant MOV_L_AT_R0_RM_RN          : std_logic_vector(15 downto 0) := "0000-----1110";
58     constant MOV_B_R0_AT_DISP_GBR       : std_logic_vector(15 downto 0) := "11000000-----";
59     constant MOV_W_R0_AT_DISP_GBR       : std_logic_vector(15 downto 0) := "11000001-----";
60     constant MOV_L_R0_AT_DISP_GBR       : std_logic_vector(15 downto 0) := "11000010-----";
61     constant MOV_B_AT_DISP_GBR_R0       : std_logic_vector(15 downto 0) := "11000100-----";
62     constant MOV_W_AT_DISP_GBR_R0       : std_logic_vector(15 downto 0) := "11000101-----";
63     constant MOV_L_AT_DISP_GBR_R0       : std_logic_vector(15 downto 0) := "11000110-----";
64     constant MOVA_AT_DISP_PC_R0        : std_logic_vector(15 downto 0) := "11000111-----";
65     constant MOV_T_RN                  : std_logic_vector(15 downto 0) := "0000----00101001";

```

```

66      constant SWAP_B_RM_RN      : std_logic_vector(15 downto 0) := "0110-----1000";
67      constant SWAP_W_RM_RN      : std_logic_vector(15 downto 0) := "0110-----1001";
68      constant XTRCT_RM_RN       : std_logic_vector(15 downto 0) := "0010-----1101";
69      constant ADD_RM_RN         : std_logic_vector(15 downto 0) := "0011-----1100";
70      constant ADD_IMM_IMM_RN    : std_logic_vector(15 downto 0) := "0111-----";
71      constant ADDC_RM_RN        : std_logic_vector(15 downto 0) := "0011-----1110";
72      constant ADDV_RM_RN        : std_logic_vector(15 downto 0) := "0011-----1111";
73      constant CMP_EQ_RM_RN      : std_logic_vector(15 downto 0) := "0011-----0000";
74      constant CMP_EQ_IMM_IMM_R0 : std_logic_vector(15 downto 0) := "10001000-----";
75      constant CMP_GE_RM_RN      : std_logic_vector(15 downto 0) := "0011-----0011";
76      constant CMP_GT_RM_RN      : std_logic_vector(15 downto 0) := "0011-----0111";
77      constant CMP_HI_RM_RN      : std_logic_vector(15 downto 0) := "0011-----0110";
78      constant CMP_HS_RM_RN      : std_logic_vector(15 downto 0) := "0011-----0010";
79      constant CMP_PL_RN         : std_logic_vector(15 downto 0) := "0100---00010101";
80      constant CMP_PZ_RN         : std_logic_vector(15 downto 0) := "0100---00010001";
81      constant CMP_STR_RM_RN     : std_logic_vector(15 downto 0) := "0010-----1100";
82      constant EXTS_B_RM_RN      : std_logic_vector(15 downto 0) := "0110-----1110";
83      constant EXTS_W_RM_RN      : std_logic_vector(15 downto 0) := "0110-----1111";
84      constant EXTU_B_RM_RN      : std_logic_vector(15 downto 0) := "0110-----1100";
85      constant EXTU_W_RM_RN      : std_logic_vector(15 downto 0) := "0110-----1101";
86      constant SUB_RM_RN         : std_logic_vector(15 downto 0) := "0011-----1000";
87      constant SUBC_RM_RN        : std_logic_vector(15 downto 0) := "0011-----1010";
88      constant SUBV_RM_RN        : std_logic_vector(15 downto 0) := "0011-----1011";
89      constant NEG_RM_RN         : std_logic_vector(15 downto 0) := "0110-----1011";
90      constant NEGC_RM_RN        : std_logic_vector(15 downto 0) := "0110-----1010";
91      constant DT_RN            : std_logic_vector(15 downto 0) := "0100---00010000";
92      constant AND_RM_RN         : std_logic_vector(15 downto 0) := "0010-----1001";
93      constant AND_IMM_IMM_R0    : std_logic_vector(15 downto 0) := "11001001-----";
94      constant AND_B_IMM_IMM_AT_R0_GBR : std_logic_vector(15 downto 0) := "11001101-----";
95      constant OR_RM_RN          : std_logic_vector(15 downto 0) := "0010-----1011";
96      constant OR_IMM_IMM_R0     : std_logic_vector(15 downto 0) := "11001011-----";
97      constant OR_B_IMM_IMM_AT_R0_GBR : std_logic_vector(15 downto 0) := "11001111-----";
98      constant XOR_RM_RN         : std_logic_vector(15 downto 0) := "0010-----1010";
99      constant XOR_IMM_IMM_R0    : std_logic_vector(15 downto 0) := "11001010-----";
100     constant XOR_B_IMM_IMM_AT_R0_GBR : std_logic_vector(15 downto 0) := "11001110-----";
101     constant NOT_RM_RN         : std_logic_vector(15 downto 0) := "0110-----0111";
102     constant TST_RM_RN         : std_logic_vector(15 downto 0) := "0010-----1000";
103     constant TST_IMM_IMM_R0    : std_logic_vector(15 downto 0) := "11001000-----";
104     constant TST_B_IMM_IMM_AT_R0_GBR : std_logic_vector(15 downto 0) := "11001100-----";
105     constant TAS_B_AT_RN       : std_logic_vector(15 downto 0) := "0100---00011011";
106     constant ROTL_RN           : std_logic_vector(15 downto 0) := "0100---00000100";
107     constant ROTR_RN           : std_logic_vector(15 downto 0) := "0100---00000101";
108     constant ROTCL_RN          : std_logic_vector(15 downto 0) := "0100---00100100";
109     constant ROTCR_RN          : std_logic_vector(15 downto 0) := "0100---00100101";
110     constant SHLL_RN           : std_logic_vector(15 downto 0) := "0100---00000000";
111     constant SHLR_RN           : std_logic_vector(15 downto 0) := "0100---00000001";
112     constant SHAL_RN           : std_logic_vector(15 downto 0) := "0100---00100000";
113     constant SHAR_RN           : std_logic_vector(15 downto 0) := "0100---00100001";
114     constant SHLL2_RN          : std_logic_vector(15 downto 0) := "0100---00001000";
115     constant SHLL8_RN          : std_logic_vector(15 downto 0) := "0100---00011000";
116     constant SHLL16_RN         : std_logic_vector(15 downto 0) := "0100---00101000";
117     constant SHLR2_RN          : std_logic_vector(15 downto 0) := "0100---00001001";
118     constant SHLR8_RN          : std_logic_vector(15 downto 0) := "0100---00011001";
119     constant SHLR16_RN         : std_logic_vector(15 downto 0) := "0100---00101001";
120     constant BT_LABEL          : std_logic_vector(15 downto 0) := "10001001-----";
121     constant BF_LABEL          : std_logic_vector(15 downto 0) := "10001011-----";
122     constant BT_S_LABEL        : std_logic_vector(15 downto 0) := "10001101-----";
123     constant BF_S_LABEL        : std_logic_vector(15 downto 0) := "10001111-----";
124     constant BRA_LABEL         : std_logic_vector(15 downto 0) := "1010-----";
125     constant BRAF_RM           : std_logic_vector(15 downto 0) := "0000---00100011";
126     constant BSR_LABEL         : std_logic_vector(15 downto 0) := "1011-----";
127     constant BSRF_RM           : std_logic_vector(15 downto 0) := "0000---00000011";
128     constant JMP_AT_RM         : std_logic_vector(15 downto 0) := "0100---00101011";
129     constant JSR_AT_RM         : std_logic_vector(15 downto 0) := "0100---00001011";
130     constant RTS               : std_logic_vector(15 downto 0) := "0000000000001011";
131     constant CLRT              : std_logic_vector(15 downto 0) := "0000000000001000";
132     constant SETT              : std_logic_vector(15 downto 0) := "0000000000011000";

```

```

133 constant NOP : std_logic_vector(15 downto 0) := "0000000000001001";
134 constant LDC_RM_GBR : std_logic_vector(15 downto 0) := "0100----00011110";
135 constant LDC_RM_VBR : std_logic_vector(15 downto 0) := "0100----00101110";
136 constant LDC_L_AT_RM_PLUS_GBR : std_logic_vector(15 downto 0) := "0100----00010111";
137 constant LDC_L_AT_RM_PLUS_VBR : std_logic_vector(15 downto 0) := "0100----00100111";
138 constant LDC_L_AT_RM_PLUS_SR : std_logic_vector(15 downto 0) := "0100----00000111";
139 constant STC_SR_RN : std_logic_vector(15 downto 0) := "0000----00000010";
140 constant STC_GBR_RN : std_logic_vector(15 downto 0) := "0000----00010010";
141 constant STC_VBR_RN : std_logic_vector(15 downto 0) := "0000----00100010";
142 constant STC_L_SR_AT_MINUSRN : std_logic_vector(15 downto 0) := "0100----00000011";
143 constant STC_L_GBR_AT_MINUSRN : std_logic_vector(15 downto 0) := "0100----00010011";
144 constant STC_L_VBR_AT_MINUSRN : std_logic_vector(15 downto 0) := "0100----00100011";
145 constant LDS_RM_PR : std_logic_vector(15 downto 0) := "0100----00101010";
146 constant LDS_L_AT_RM_PLUS_PR : std_logic_vector(15 downto 0) := "0100----00100110";
147 constant STS_PR_RN : std_logic_vector(15 downto 0) := "0000----00101010";
148 constant STS_L_PR_AT_MINUSRN : std_logic_vector(15 downto 0) := "0100----00100010";
149
150 -- =====
151 -- ControlSH2 common control signal constants
152 -- =====
153
154 -- reg_ALU_IN [2(reg_in_src) | 4(reg_in_sel) | 1(reg_store)]
155 constant REG_IN_NONE : std_logic_vector(7 downto 0) := "00000000"; -- no reg write
156 constant REG_IN_R0_ONLY : std_logic_vector(7 downto 0) := "00100001"; -- write R0 src=datadatabus
157 constant REG_IN_R0_REGFILE : std_logic_vector(7 downto 0) := "00000001"; -- write R0, src=regfile through ALU?
or ALU
158
159 -- reg_MAU [4(reg_a_sel) | 4(reg_b_sel) | 4(reg_ax_sel) | 1(reg_ax_store) | 4(reg_a2_sel)]
160 constant REG_MAU_NONE : std_logic_vector(16 downto 0) := "0000000000000000";
161
162 -- alu_in [1(alu_sel_a) + 1(alu_sel_b)]
163 constant ALU_IN_DATAREG : std_logic_vector(1 downto 0) := "00"; -- A=datadatabus B=reg
164 constant ALU_IN_DATAIMM : std_logic_vector(1 downto 0) := "01"; -- A=datadatabus B=immediate
165 constant ALU_IN_REGREG : std_logic_vector(1 downto 0) := "10"; -- A=reg B=reg
166 constant ALU_IN_REGIMM : std_logic_vector(1 downto 0) := "11"; -- A=reg, B=immediate
167
168 -- alu_gen [4(f_cmd) | 2(c_in_cmd) | 3(s_cmd) | 2(alu_cmd)]
169 constant ALU_GEN_PASS : std_logic_vector(10 downto 0) := "1100000000"; -- pass / NOP
170 constant ALU_GEN_IMM : std_logic_vector(10 downto 0) := "1010000000"; -- load immediate
171 constant ALU_GEN_ADD : std_logic_vector(10 downto 0) := "1010000001"; -- ADD
172 constant ALU_GEN_ADDC : std_logic_vector(10 downto 0) := "1010100001"; -- ADDC
173 constant ALU_GEN_SUB : std_logic_vector(10 downto 0) := "01010100001"; -- SUB / CMP
174 constant ALU_GEN_SUBC : std_logic_vector(10 downto 0) := "01011100001"; -- SUBC
175 constant ALU_GEN_AND : std_logic_vector(10 downto 0) := "1000000000"; -- AND
176 constant ALU_GEN_OR : std_logic_vector(10 downto 0) := "1110000000"; -- OR
177 constant ALU_GEN_XOR : std_logic_vector(10 downto 0) := "0110000000"; -- XOR
178 constant ALU_GEN_NOT : std_logic_vector(10 downto 0) := "0011000000"; -- NOT
179 constant ALU_GEN_NEG : std_logic_vector(10 downto 0) := "01010100001"; -- NEG (NOT B, cin=1, adder)
180 constant ALU_GEN_NEGC : std_logic_vector(10 downto 0) := "01011100001"; -- NEG (NOT B, cin=~C, adder)
181 constant ALU_GEN_SUBV : std_logic_vector(10 downto 0) := "01010100001"; -- SUBV
182 constant ALU_GEN_CMP_STR : std_logic_vector(10 downto 0) := "0110000000"; -- CMP/STR (XOR)
183 constant ALU_GEN_SHLL1 : std_logic_vector(10 downto 0) := "1100000010"; -- shift left 1 (SHLL/SHAL)
184 constant ALU_GEN_SHLR1_L : std_logic_vector(10 downto 0) := "11000010010"; -- shift right 1 logical (SHLR)
185 constant ALU_GEN_SHLR1_A : std_logic_vector(10 downto 0) := "11000010110"; -- shift right 1 arith (SHAR)
186 constant ALU_GEN_SHLR_N : std_logic_vector(10 downto 0) := "11000010000"; -- SHLRn (2/8/16)
187 constant ALU_GEN_ROTLL : std_logic_vector(10 downto 0) := "11000001010"; -- ROTLL
188 constant ALU_GEN_ROTTR : std_logic_vector(10 downto 0) := "11000011010"; -- ROTTR
189 constant ALU_GEN_ROTCL : std_logic_vector(10 downto 0) := "11001001110"; -- ROTCL
190 constant ALU_GEN_ROTCTR : std_logic_vector(10 downto 0) := "11001011110"; -- ROTCTR
191 constant ALU_GEN_NONE : std_logic_vector(10 downto 0) := "0000000000"; -- no ALU op
192
193 -- alu_sh2 [2(shift_sel) + 2(extend_sel) + 3(res_sel) + 4(t_sel)]
194 constant ALU_SH2_NONE : std_logic_vector(10 downto 0) := "0000000000"; -- no fancy sh2 select generic
195
196 -- pau_controls [2(pau_src_sel) | 2(pau_offset_sel) | 2(pr_sel)]
197 constant PAU_HOLD : std_logic_vector(5 downto 0) := "000000"; -- hold (no fetch)
198 constant PAU_SEQ : std_logic_vector(5 downto 0) := "001100"; -- sequential fetch (PC+2)

```

```

199     constant PAU_BRANCH_DISP      : std_logic_vector(5 downto 0) := "110000"; -- PC+4 relative branch
200     constant PAU_BRANCH_REG       : std_logic_vector(5 downto 0) := "110100"; -- BRAF Rm (PC+4 + Rm)
201     constant PAU_BSR              : std_logic_vector(5 downto 0) := "110001"; -- BSR (PC+4 base, save PR)
202     constant PAU_BSRF             : std_logic_vector(5 downto 0) := "110101"; -- BSRF Rm (PC+4 + Rm, save PR)
203     constant PAU_JMP              : std_logic_vector(5 downto 0) := "101000"; -- JMP @Rm
204     constant PAU_JSR              : std_logic_vector(5 downto 0) := "101001"; -- JSR @Rm (save PR)
205     constant PAU_RTS              : std_logic_vector(5 downto 0) := "001100"; -- RTS (restore PR)
206     constant PAU_LDS_PR           : std_logic_vector(5 downto 0) := "001011"; -- LDS.L @Rm,PR (pr_sel=3: load PR from
data bus)
207
208     -- dau_controls [2(dau_src_sel) + 2(dau_shift_sel) + 2(dau_offset_sel) + 1(dau_inc_dec_sel) + 2(dau_inc_dec_bit)
+ 1(dau_pre_post_sel)]
209     constant DAU_CTRL_NONE        : std_logic_vector(9 downto 0) := "0000100000"; -- no DAU operation / pass
through source
210     constant DAU_CTRL_OFFSET_DISP8 : std_logic_vector(9 downto 0) := "0000000000"; -- use 8-bit displacement as
offset
211
212     -- dau_write [1(gbr_write) | 1(vbr_write)]
213     constant DAU_WRITE_NONE       : std_logic_vector(1 downto 0) := "00";
214     constant DAU_WRITE_GBR       : std_logic_vector(1 downto 0) := "10";
215     constant DAU_WRITE_VBR       : std_logic_vector(1 downto 0) := "01";
216
217     -- RE / WE byte-enable patterns (active-low: '0' = enabled)
218     constant RE_NONE             : std_logic_vector(3 downto 0) := "1111"; -- no read
219     constant RE_BYTE            : std_logic_vector(3 downto 0) := "1110"; -- byte read
220     constant RE_WORD            : std_logic_vector(3 downto 0) := "1100"; -- word read
221     constant RE_LONG            : std_logic_vector(3 downto 0) := "0000"; -- longword read
222     constant WE_NONE            : std_logic_vector(3 downto 0) := "1111"; -- no write
223     constant WE_BYTE            : std_logic_vector(3 downto 0) := "1110"; -- byte write
224     constant WE_WORD            : std_logic_vector(3 downto 0) := "1100"; -- word write
225     constant WE_LONG            : std_logic_vector(3 downto 0) := "0000"; -- longword write
226
227     -- =====
228     -- Index constants for instruction fields (for unpacking in CPU)
229     -- =====
230     -- for reg_ALU_IN [3(reg_in_src) + 4(reg_in_sel) + 1(reg_store)] MSB-first
231     constant REG_IN_SRC_IDX      : integer := 7;
232     constant REG_IN_SEL_IDX      : integer := 4;
233     constant REG_STORE_IDX       : integer := 0;
234
235     -- for reg_MAU [4(reg_a_sel) + 4(reg_b_sel) + 4(reg_ax_sel) + 1(reg_ax_store) + 4(reg_a2_sel)] MSB-first
236     constant REG_MAU_A_SEL_IDX   : integer := 16;
237     constant REG_MAU_B_SEL_IDX   : integer := 12;
238     constant REG_MAU_AX_SEL_IDX  : integer := 8;
239     constant REG_MAU_AX_STORE_IDX : integer := 4;
240     constant REG_MAU_A2_SEL_IDX  : integer := 3;
241
242     -- for alu_in [1(alu_sel_a) + 1(alu_sel_b)] MSB-first
243     constant ALU_SEL_A_IDX       : integer := 1;
244     constant ALU_SEL_B_IDX       : integer := 0;
245
246     -- for alu_gen [4(f_cmd) + 2(c_in_cmd) + 3(s_cmd) + 2(alu_cmd)] MSB-first
247     constant ALU_F_CMD_IDX       : integer := 10;
248     constant ALU_C_IN_CMD_IDX    : integer := 6;
249     constant ALU_S_CMD_IDX       : integer := 4;
250     constant ALU_ALU_CMD_IDX     : integer := 1;
251
252     -- for alu_sh2 [2(shift_sel) + 2(extend_sel) + 3(res_sel) + 4(t_sel)] MSB-first
253     constant ALU_SHIFT_SEL_IDX   : integer := 10;
254     constant ALU_EXTEND_SEL_IDX  : integer := 8;
255     constant ALU_RES_SEL_IDX     : integer := 6;
256     constant ALU_T_SEL_IDX       : integer := 3;
257
258     -- for dau_controls [2(dau_src_sel) + 2(dau_shift_sel) + 2(dau_offset_sel) + 1(dau_inc_dec_sel) +
2(dau_inc_dec_bit) + 1(dau_pre_post_sel)] MSB-first
259     constant DAU_SRC_SEL_IDX     : integer := 9;
260     constant DAU_SHIFT_SEL_IDX   : integer := 7;
261     constant DAU_OFFSET_SEL_IDX  : integer := 5;

```

```
262     constant DAU_INC_DEC_SEL_IDX      : integer := 3;
263     constant DAU_INC_DEC_BIT_IDX      : integer := 2;
264     constant DAU_PRE_POST_SEL_IDX     : integer := 0;
265
266     -- for dau_write [1(gbr_write) + 1(vbr_write)] MSB-first
267     constant DAU_WRITE_GBR_IDX        : integer := 1;
268     constant DAU_WRITE_VBR_IDX        : integer := 0;
269
270     -- for pau_controls [2(pau_src_sel) + 2(pau_offset_sel) + 2(pr_sel)] MSB-first
271     constant PAU_SRC_SEL_IDX          : integer := 5;
272     constant PAU_OFFSET_SEL_IDX       : integer := 3;
273     constant PAU_PR_SEL_IDX           : integer := 1;
274
275 end InstructionConstants;
276
```